

Data Types: Logicals & Numbers

PSY 410: Data Science for Psychology

Dr. Sara Weston

2026-05-06

Why data types matter

A published error

A participant rates these three items from the Rosenberg Self-Esteem Scale on a 1–5 scale (1 = Strongly Disagree, 5 = Strongly Agree):

#	Item	Resp.
1	I feel that I am a person of worth, at least on an equal plane with others.	4
2	I feel that I have a number of good qualities.	3
3	All in all, I am inclined to feel that I am a failure.	2

What's their self-esteem score so far?

If you said **9**, you forgot that item 3 is reverse-coded — a **2** should become **(5 + 1) – 2 = 4**, making the real total **11**. Mistakes like this happen in published papers, and they live in how you handle data types.

Everything in R is a type

In R, every value has a **type**:

- "hello" is a **string** (character)
- 42 is a **number** (double)
- TRUE is a **logical** (boolean)
- "Female" can be a **factor** (categorical)

Understanding types helps you choose the right functions, avoid cryptic errors, and transform data correctly.

Today's focus

We'll dive into two fundamental types:

1. **Logical vectors** — TRUE/FALSE values for filtering and conditional logic
2. **Numbers** — integers and doubles for calculations and summaries

Psychology application: Computing scale scores, recoding responses, handling missing data

Logical vectors

What are logicals?

Logical vectors contain only **TRUE**, **FALSE**, or **NA**:

```
x <- c(TRUE, FALSE, TRUE, NA, FALSE)
x
```

```
[1] TRUE FALSE TRUE NA FALSE
```

They're the result of comparisons:

```
ages <- c(18, 22, 45, 17, 30)
ages >= 18 # Is each age 18 or older?
```

```
[1] TRUE TRUE TRUE FALSE TRUE
```

Comparison operators (review)

Operator	Meaning
<code>==</code>	Equal to
<code>!=</code>	Not equal to
<code><</code>	Less than
<code>></code>	Greater than
<code><=</code>	Less than or equal to
<code>>=</code>	Greater than or equal to
<code>%in%</code>	Is in a set

Logical operators

Combine comparisons with **Boolean operators**:

Operator	Meaning	Example
&	AND	<code>age >= 18 & age < 65</code>
	OR	<code>diagnosis == "Depression" diagnosis == "Anxiety"</code>
!	NOT	<code>!is.na(response)</code>

AND vs OR

```
age <- c(17, 22, 45, 70)
diagnosis <- c("Depression", "Anxiety", "Other", "Depression")

# AND: Both conditions must be TRUE
age >= 18 & age < 65
```

```
[1] FALSE TRUE TRUE FALSE
```

```
# OR: At least one condition must be TRUE
diagnosis == "Depression" | diagnosis == "Anxiety"
```

```
[1] TRUE TRUE FALSE TRUE
```

Using logicals in filter()

```
survey_data <- tibble(  
  id = 1:5,  
  age = c(17, 22, 45, 70, 30),  
  consent = c(FALSE, TRUE, TRUE, TRUE, TRUE),  
  depression = c(10, 25, 18, 12, 30)  
)  
  
# Keep only consenting adults with high depression  
survey_data |>  
  filter(consent & age >= 18 & depression >= 20)
```

```
# A tibble: 2 × 4  
   id    age consent depression  
<int> <dbl> <lgl>      <dbl>  
1     2    22  TRUE         25  
2     5    30  TRUE         30
```

any() and all()

Check if **any** or **all** values are TRUE:

```
responses <- c(TRUE, FALSE, FALSE, TRUE)
```

```
any(responses)  # Is at least one TRUE?
```

```
[1] TRUE
```

```
all(responses)  # Are all TRUE?
```

```
[1] FALSE
```

Useful for checking data quality:

```
# Did anyone fail the attention check?
```

```
any(survey_data$consent == FALSE)
```

```
[1] TRUE
```

Counting with logicals

Remember: **TRUE = 1** and **FALSE = 0**

```
# How many adults?  
sum(survey_data$age >= 18)
```

```
[1] 4
```

```
# What proportion are adults?  
mean(survey_data$age >= 18)
```

```
[1] 0.8
```

Conditional values

if_else(): Two-way decisions

`if_else()` creates new values based on a condition:

```
survey_data |>
  mutate(
    age_group = if_else(age >= 18, "Adult", "Minor")
  )
```

```
# A tibble: 5 × 5
   id   age consent depression age_group
<int> <dbl> <lgl>         <dbl> <chr>
1     1    17 FALSE           10 Minor
2     2    22  TRUE           25 Adult
3     3    45  TRUE           18 Adult
4     4    70  TRUE           12 Adult
5     5    30  TRUE           30 Adult
```

Syntax: `if_else(condition, value_if_true, value_if_false)`

Handling NA with if_else()

By default, `if_else()` keeps **NA** values:

```
responses <- c(1, 2, NA, 4, 5)

if_else(responses >= 3, "High", "Low")
```

```
[1] "Low"  "Low"  NA      "High" "High"
```

You can specify what to do with **NA**:

```
if_else(responses >= 3, "High", "Low", missing = "No response")
```

```
[1] "Low"          "Low"          "No response" "High"         "High"
```


case_when(): Multiple conditions

For more than two outcomes, use `case_when()`:

```
survey_data |>
  mutate(
    depression_category = case_when(
      depression < 14 ~ "Minimal",
      depression < 20 ~ "Mild",
      depression < 29 ~ "Moderate",
      depression >= 29 ~ "Severe"
    )
  )
```

A tibble: 5 × 5

	id	age	consent	depression	depression_category
	<int>	<dbl>	<lgl>	<dbl>	<chr>
1	1	17	FALSE	10	Minimal
2	2	22	TRUE	25	Moderate
3	3	45	TRUE	18	Mild
4	4	70	TRUE	12	Minimal
5	5	30	TRUE	30	Severe

case_when() rules

- Conditions are checked **in order**
- The first **TRUE** condition wins
- If no condition matches, you get **NA**

Always include a catch-all:

```
case_when(  
  age < 18 ~ "Minor",  
  age < 65 ~ "Adult",  
  age >= 65 ~ "Senior",  
  .default = NA # Explicit about NAs  
)
```

if_else() vs case_when() — when to use which

Situation	Use
Two outcomes (yes/no, pass/fail)	<code>if_else()</code>
Three or more categories	<code>case_when()</code>
Recoding a Likert scale into groups	<code>case_when()</code>
Flagging a single condition	<code>if_else()</code>

When in doubt, start with `if_else()`. Graduate to `case_when()` when you need more categories.

Psychology example: Reverse coding

Rosenberg Self-Esteem Scale

Please record the appropriate answer for each item, depending on whether you Strongly agree, agree, disagree, or strongly disagree with it.

- 1 = Strongly agree
- 2 = Agree
- 3 = Disagree
- 4 = Strongly disagree

- ___ 1. On the whole, I am satisfied with myself.
- ___ 2. At times I think I am no good at all.
- ___ 3. I feel that I have a number of good qualities.
- ___ 4. I am able to do things as well as most other people.
- ___ 5. I feel I do not have much to be proud of.
- ___ 6. I certainly feel useless at times.
- ___ 7. I feel that I'm a person of worth.
- ___ 8. I wish I could have more respect for myself.
- ___ 9. All in all, I am inclined to think that I am a failure.
- ___ 10. I take a positive attitude toward myself.

Rosenberg Self-Esteem Scale showing 10 items with Likert response options — items 2, 5, 6, 8, and 9 are reverse-coded

Psychology example: Reverse coding

Many scales have reverse-coded items:

```
# Original responses (1-5 scale)
rosenberg <- tibble(
  id = 1:3,
  item1 = c(5, 4, 3), # Regular item
  item2 = c(2, 3, 4)  # Reverse-coded
)

# Reverse code item2
rosenberg = rosenberg |>
  mutate(
    item2_reversed = case_when(
      item2 == 1 ~ 5,
      item2 == 2 ~ 4,
      item2 == 3 ~ 3,
      item2 == 4 ~ 2,
      item2 == 5 ~ 1
    )
  )
```

rosenberg

```
# A tibble: 3 × 4
  id item1 item2 item2_reversed
<int> <dbl> <dbl>         <dbl>
1     1     5     2             4
2     2     4     3             3
3     3     3     4             2
```

Easier reverse coding

For scales, use arithmetic:

```
rosenberg |>
  mutate(
    item2_reversed = 6 - item2 # For 1-5 scale: 6 - x
  )
```

```
# A tibble: 3 × 4
   id item1 item2 item2_reversed
<int> <dbl> <dbl>         <dbl>
1     1     5     2             4
2     2     4     3             3
3     3     3     4             2
```

General formula: $(\text{max} + \text{min}) - \text{original_value}$

- 1-5 scale: $6 - x$ (because $1 + 5 = 6$)
- 1-7 scale: $8 - x$ (because $1 + 7 = 8$)

Pair coding break

Your turn: Recode responses

You have survey data with a 1-7 attention check item where the correct answer is 4:

```
attention_data <- tibble(  
  participant_id = 1:6,  
  attention_check = c(4, 3, 4, 7, NA, 4)  
)
```

1. Create a new column **passed** that is **TRUE** if they answered 4, **FALSE** otherwise
2. Create a column **status** with three values: “Passed”, “Failed”, or “No response” (for NA)
3. What proportion of participants passed?

Time: 10 minutes

Before we move on

 [Submit your code on Canvas](#) for participation credit. Paste what you have — it doesn't need to work perfectly.

Numbers

Types of numbers

R distinguishes two numeric types:

- **Integers:** whole numbers (1, 2, 3)
- **Doubles:** numbers with decimals (1.5, 2.718, 3.14159)

Most of the time, R uses doubles automatically:

```
typeof(42)
```

```
[1] "double"
```

```
typeof(42L) # The L forces it to be an integer
```

```
[1] "integer"
```

You rarely need to worry about this distinction!

Rounding

```
reaction_times <- c(245.678, 198.234, 312.891)
```

```
round(reaction_times)      # Round to nearest integer
```

```
[1] 246 198 313
```

```
round(reaction_times, 1)   # Round to 1 decimal place
```

```
[1] 245.7 198.2 312.9
```

```
floor(reaction_times)      # Round down
```

```
[1] 245 198 312
```

```
ceiling(reaction_times)    # Round up
```

```
[1] 246 199 313
```

Summary functions (review)

Common calculations you've been using:

```
scores <- c(12, 18, 25, 22, 30, 15, NA)
```

```
mean(scores, na.rm = TRUE)
```

```
[1] 20.33333
```

```
median(scores, na.rm = TRUE)
```

```
[1] 20
```

```
sd(scores, na.rm = TRUE)
```

```
[1] 6.65332
```

```
min(scores, na.rm = TRUE)
```

```
[1] 12
```

```
max(scores, na.rm = TRUE)
```

The na.rm argument

Most summary functions need `na.rm = TRUE` to handle missing data:

```
scores <- c(12, 18, NA, 22, 30)
```

```
mean(scores)           # Returns NA
```

```
[1] NA
```

```
mean(scores, na.rm = TRUE) # Ignores NA
```

```
[1] 20.5
```

Counting non-missing values

```
scores <- c(12, 18, NA, 22, 30, NA)

sum(!is.na(scores)) # Count non-missing
```

```
[1] 4
```

Or in a summary:

```
tibble(scores) |>
  summarize(
    n = sum(!is.na(scores)),
    mean_score = mean(scores, na.rm = TRUE)
  )
```

```
# A tibble: 1 × 2
      n mean_score
<int>     <dbl>
1     4      20.5
```

Computing scale scores

Real psychology task: Scale scoring

You've collected survey data with multiple items per scale:

```
scale_data <- tibble(  
  id = 1:4,  
  anxiety1 = c(3, 2, 4, NA),  
  anxiety2 = c(4, 3, 5, 2),  
  anxiety3 = c(3, 2, 4, 1),  
  anxiety4 = c(4, 3, NA, 2)  
)
```

scale_data

```
# A tibble: 4 × 5  
   id anxiety1 anxiety2 anxiety3 anxiety4  
  <int>   <dbl>   <dbl>   <dbl>   <dbl>  
1     1     3     4     3     4  
2     2     2     3     2     3  
3     3     4     5     4    NA  
4     4    NA     2     1     2
```

How do you compute a total or mean score?

Option 1: Manual addition

```
scale_data |>
  mutate(
    anxiety_total = anxiety1 + anxiety2 + anxiety3 + anxiety4
  )
```

A tibble: 4 × 6

	id	anxiety1	anxiety2	anxiety3	anxiety4	anxiety_total
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	1	3	4	3	4	14
2	2	2	3	2	3	10
3	3	4	5	4	NA	NA
4	4	NA	2	1	2	NA

Problem: If any item is **NA**, the whole sum is **NA**!

A new tool: `rowwise()`

By default, `mutate()` works **column-wise** — `sum()` and `mean()` inside it see entire columns, not individual rows.

For scale scoring we need the opposite: combine values *across* columns *within* a single participant.

`rowwise()` switches `mutate()` into **row-by-row mode**:

```
data |>
  rowwise() |>      # switch to row-by-row mode
  mutate(...) |>    # now sees one row at a time
  ungroup()         # switch back when done
```

Why rowwise() matters

Without it, `sum()` collapses across the whole column — the same number for every row:

```
scale_data |>
  mutate(
    anxiety_total = sum(c(anxiety1, anxiety2, anxiety3, anxiety4),
                        na.rm = TRUE)
  )
```

A tibble: 4 × 6

	id	anxiety1	anxiety2	anxiety3	anxiety4	anxiety_total
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	1	3	4	3	4	42
2	2	2	3	2	3	42
3	3	4	5	4	NA	42
4	4	NA	2	1	2	42

That's not what we want.

Option 2: Sum with na.rm + rowwise()

Now combine `sum(..., na.rm = TRUE)` with `rowwise()`:

```
scale_data |>
  rowwise() |>
  mutate(
    anxiety_total = sum(c(anxiety1, anxiety2, anxiety3, anxiety4),
                        na.rm = TRUE)
  ) |>
  ungroup()
```

A tibble: 4 × 6

	id	anxiety1	anxiety2	anxiety3	anxiety4	anxiety_total
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	1	3	4	3	4	14
2	2	2	3	2	3	10
3	3	4	5	4	NA	13
4	4	NA	2	1	2	5

Computing mean scores

```
scale_data |>
  rowwise() |>
  mutate(
    anxiety_mean = mean(c(anxiety1, anxiety2, anxiety3, anxiety4),
                        na.rm = TRUE)
  ) |>
  ungroup()
```

A tibble: 4 × 6

	id	anxiety1	anxiety2	anxiety3	anxiety4	anxiety_mean
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	1	3	4	3	4	3.5
2	2	2	3	2	3	2.5
3	3	4	5	4	NA	4.33
4	4	NA	2	1	2	1.67



Tip

Mean vs Total: Use means when participants might have different numbers of items answered.

Cleaner approach: pivot then summarize

```
scale_data |>
  pivot_longer(
    cols = starts_with("anxiety"),
    names_to = "item",
    values_to = "response"
  ) |>
  group_by(id) |>
  summarize(
    anxiety_mean = mean(response, na.rm = TRUE),
    anxiety_total = sum(response, na.rm = TRUE),
    n_items = sum(!is.na(response))
  )
```


Cleaner approach: pivot then summarize

```
# A tibble: 4 × 4
  id anxiety_mean anxiety_total n_items
<int>      <dbl>      <dbl>    <int>
1     1         3.5         14        4
2     2         2.5         10        4
3     3         4.33        13        3
4     4         1.67         5         3
```

When to worry about missing items

Should you compute a scale score if someone only answered 1 out of 4 items?

Common rules:

- **Require $\geq 50\%$** of items answered
- **Require $\geq 75\%$** for critical scales
- **Document your decision** clearly

Implementing a missingness rule

```
scale_data |>
  rowwise() |>
  mutate(
    n_answered = sum(!is.na(c(anxiety1, anxiety2, anxiety3, anxiety4))),
    anxiety_mean = if_else(
      n_answered >= 3, # At least 3 of 4 items
      mean(c(anxiety1, anxiety2, anxiety3, anxiety4), na.rm = TRUE),
      NA_real_ # NA if too many missing
    )
  ) |>
  ungroup()
```

A tibble: 4 × 7

	id	anxiety1	anxiety2	anxiety3	anxiety4	n_answered	anxiety_mean
	<int>	<dbl>	<dbl>	<dbl>	<dbl>	<int>	<dbl>
1	1	3	4	3	4	4	3.5
2	2	2	3	2	3	4	2.5
3	3	4	5	4	NA	3	4.33
4	4	NA	2	1	2	3	1.67

More complex scales: Subscales

Some measures have multiple subscales:

```
dass_data <- tibble(  
  id = 1:3,  
  # Depression items  
  dass_d1 = c(2, 3, 1), dass_d2 = c(3, 3, 2),  
  # Anxiety items  
  dass_a1 = c(1, 4, 2), dass_a2 = c(2, 4, 3),  
  # Stress items  
  dass_s1 = c(3, 2, 4), dass_s2 = c(4, 3, 4)  
)
```

Computing subscales

```
dass_data |>
  rowwise() |>
  mutate(
    depression = mean(c(dass_d1, dass_d2), na.rm = TRUE),
    anxiety = mean(c(dass_a1, dass_a2), na.rm = TRUE),
    stress = mean(c(dass_s1, dass_s2), na.rm = TRUE)
  ) |>
  ungroup() |>
  select(id, depression, anxiety, stress)
```

Computing subscales

```
# A tibble: 3 × 4
  id depression anxiety stress
<int>      <dbl>   <dbl>   <dbl>
1     1        2.5     1.5     3.5
2     2         3       4       2.5
3     3        1.5     2.5     4
```

Wrapping up

Key takeaways

1. **Logical vectors** (TRUE/FALSE) are powerful for filtering and conditions
2. **if_else()** for two-way decisions, **case_when()** for multiple outcomes
3. **Reverse coding:** $(\text{max} + \text{min}) - \text{value}$
4. **Scale scoring:**
 - Use `rowwise()` + `sum()/mean()` with `na.rm = TRUE`
 - Or pivot longer then group and summarize
 - Decide how to handle missing items
5. **Always document** your scoring decisions

Before next class

Read:

- R4DS Ch 14: Strings (sections 14.1–14.3 only)
- R4DS Ch 16: Factors

Do:

- Submit Assignment 5
- Practice computing scale scores with your own data
- Think about categorical variables in your final project

The one thing to remember

Scoring a scale correctly is the most common data task in psychology — and the easiest place to introduce errors.

See you Monday for strings and factors!

On your own

Practice: Score the PHQ-9

The PHQ-9 is a 9-item depression screener (0-3 scale):

```
phq_data <- tibble(
  participant = 1:5,
  phq1 = c(2, 1, 3, 0, 2),
  phq2 = c(2, 0, 3, 1, NA),
  phq3 = c(1, 1, 2, 0, 2),
  phq4 = c(2, NA, 3, 0, 1),
  phq5 = c(1, 0, 3, 1, 2),
  phq6 = c(2, 1, 2, 0, 3),
  phq7 = c(1, 1, 3, 1, 2),
  phq8 = c(2, 0, 2, 0, 1),
  phq9 = c(1, 1, 3, 0, 2)
)
```

1. Compute a **total score** (sum of all 9 items)
2. Only compute the score if **at least 7 items** are answered
3. Create a **severity category**: Minimal (0-4), Mild (5-9), Moderate (10-14), Moderately Severe (15-19), Severe (20-27)

